# Ingest qualifying json files

```
dbutils.widgets.text("p_data_source", "")
v_data_source = dbutils.widgets.get("p_data_source")
```

```
dbutils.widgets.text("p_file_date", "2021-03-21")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

## Step 1 - Read the JSON file using the spark dataframe reader API

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType
```

```
qualifying_schema = StructType(fields=[StructField("qualifyId", IntegerType(), False),
                                       StructField("raceId", IntegerType(), True),
                                       StructField("driverId", IntegerType(), True),
                                       StructField("constructorId", IntegerType(), True),
                                       StructField("number", IntegerType(), True),
                                       StructField("position", IntegerType(), True),
                                       StructField("q1", StringType(), True),
                                       StructField("q2", StringType(), True),
                                       StructField("q3", StringType(), True),
                                      ])
```

```
qualifying_df = spark.read \
.schema(qualifying_schema) \
.option("multiLine", True) \
.json(f"{raw_folder_path}/{v_file_date}/qualifying")
```

## Step 2 - Rename columns and add new columns

- Rename qualifyingId, driverId, constructorId and raceId
- Add ingestion_date with current timestamp

```
qualifying_with_ingestion_date_df = add_ingestion_date(qualifying_df)
```

```
from pyspark.sql.functions import current_timestamp, lit
```

```
final_df = qualifying_df.withColumnRenamed("qualifyId", "qualify_id") \
.withColumnRenamed("driverId", "driver_id") \
.withColumnRenamed("raceId", "race_id") \
.withColumnRenamed("constructorId", "constructor_id") \
.withColumn("ingestion_date", current_timestamp()) \
.withColumn("data_source", lit(v_data_source)) \
.withColumn("file_date", lit(v_file_date))
```

## Step 3 - Write to output to processed container in parquet format

```
# final_df.write.mode("overwrite").format("parquet").saveAsTable("f1_processed.qualifying")
```

```
# overwrite_partition(final_df, 'f1_processed', 'qualifying', 'race_id')
```

```
merge_condition = "tgt.qualify_id = src.qualify_id AND tgt.race_id = src.race_id"
merge_delta_data(final_df, 'f1_processed', 'qualifying', processed_folder_path, merge_condition, 'race_id')
```

```
# display(spark.read.parquet(f"{processed_folder_path}/qualifying"))
```

```
%sql
select * from f1_processed.qualifying;
```

Table

| | qualify_id | race_id | driver_id | constructor_id | number | position | q1 | q2 |
|---|---|---|---|---|---|---|---|---|
| 1 | 3374 | 339 | 17 | 9 | 6 | 1 | 1:51.886 | 1:48.210 |
| 2 | 3375 | 339 | 3 | 131 | 4 | 2 | 1:52.560 | 1:47.417 |
| 3 | 3376 | 339 | 20 | 9 | 5 | 3 | 1:47.632 | 1:46.828 |
| 4 | 3377 | 339 | 16 | 10 | 14 | 4 | 1:49.479 | 1:47.085 |
| 5 | 3378 | 339 | 807 | 3 | 10 | 5 | 1:49.664 | 1:47.346 |
| 6 | 3379 | 339 | 9 | 4 | 11 | 6 | 1:46.283 | 1:46.951 |
| 7 | 3380 | 339 | 22 | 3 | 9 | 7 | 1:50.301 | 1:48.371 |
| 8 | 3381 | 339 | 30 | 131 | 3 | 8 | 1:52.239 | 1:48.400 |
| 9 | 3382 | 339 | 155 | 15 | 23 | 9 | 1:48.467 | 1:47.792 |
| 10 | 3383 | 339 | 24 | 10 | 15 | 10 | 1:49.922 | 1:48.238 |
| 11 | 3384 | 339 | 808 | 4 | 12 | 11 | 1:47.952 | 1:48.760 |
| 12 | 3385 | 339 | 37 | 15 | 22 | 12 | 1:47.153 | 1:48.771 |
| 13 | 3386 | 339 | 67 | 5 | 16 | 13 | 1:48.945 | 1:49.207 |
| 14 | 3387 | 339 | 153 | 5 | 17 | 14 | 1:48.655 | 1:49.464 |

8,734 rows

```
dbutils.notebook.exit("Success")
```

```
Success
```